

PRISM: Paradigm-Guided Reasoning with Iterative Solver Memory for Constraint Satisfaction

Anonymous submission

Abstract

Constraint Satisfaction Problems (CSPs) underpin diverse real-world applications—task planning, scheduling, and combinatorial optimization—yet Large Language Models exhibit a sharp accuracy cliff as problem difficulty grows: on the ZebraLogic benchmark, LLM accuracy approaches random performance once the underlying SMT solver encounters more than 20 conflicts. We trace this failure to two architectural gaps in existing neuro-symbolic approaches: (i) repair mechanisms receive only coarse UNSAT signals, leading to documented stagnation cycles; (ii) each problem is solved in isolation, discarding all accumulated solving experience.

We present **PRISM** (Paradigm-guided Reasoning with Iterative Solver Memory), a memory-augmented neuro-symbolic framework built on a single unifying observation: in CSP solving, every inference step is mechanically certifiable by an SMT solver. This enables formally grounded experience accumulation at two complementary granularities.

Offline, PRISM distills verified solving paradigms from successful trajectories. Each paradigm is a five-tuple encoding trigger conditions, an inference operation, and Z3-formalizable pre/post-conditions. Candidates undergo triple Z3 screening—soundness via random-state sampling, effect via non-contradiction and non-vacuousness checks, and trigger selectivity via set-intersection matching against a canonical constraint pool—before admission to the Paradigm Library. This produces a compact verified Paradigm Library from training trajectories (40:1 compression ratio).

Online, retrieved paradigms guide LLM inference while a Typed Repair Trajectory Memory records canonicalized UNSAT cores and structured repair signatures. Jaccard-based stagnation detection and cosine-based loop detection trigger a four-level adaptive escalation protocol (switch target $\rightarrow$ switch type $\rightarrow$ revert from a bounded checkpoint stack $\rightarrow$ full retranslation), with a worst-case bounded LLM-call budget of $R + 2$ per puzzle.

To our knowledge, PRISM is the first framework to apply SMT-based mechanical screening to LLM-induced reasoning rules at the paradigm level. Evaluation on ZebraLogic (3×5 to 6×6 scales, seven baselines) demonstrates consistent improvements over prior neuro-symbolic approaches across all difficulty levels, with gains increasing monotonically with problem difficulty—validating that formally-grounded experience accumulation provides compounding value for harder instances. [Note to authors: update with real performance figures before submission.]

1 Introduction

Constraint Satisfaction Problems (CSPs)—finding variable assignments that simultaneously satisfy a set of constraints—model a wide class of real-world tasks, from scheduling and resource allocation to logical reasoning. Formally, a CSP is a triple $\mathcal{P} = (\mathbf{X}, \mathbf{D}, \mathbf{C})$ where $\mathbf{X}$ is a set of variables, $\mathbf{D}$ specifies finite domains, and $\mathbf{C}$ is a set of constraints. A solution is a complete assignment θ satisfying all constraints.

Recent Large Language Models offer complementary capabilities to classical symbolic solvers: flexible natural-language understanding and multi-step inference. This motivates a hybrid approach in which an LLM formalizes natural-language constraints, a solver verifies them, and the LLM repairs failures. However, Lin et al. (2025) document a severe accuracy cliff: across all LLM-based systems, performance drops sharply from 48–60% at easy scales to 8–15% (near random) at hard 6×6 scales, with more than 20 solver conflicts as the decisive threshold. Computational scaling—Best-of- N , self-verification—provides diminishing returns beyond this threshold.

Two structural deficiencies explain this cliff. First, *repair inefficiency*: when the solver returns UNSAT, existing systems feed back only a raw error string or generic unsatisfiability statement. The LLM must repair essentially blindly, producing near-identical erroneous outputs in successive rounds—a phenomenon termed *stagnation* (Lin et al. 2025). Second, *absent cross-problem transfer*: every puzzle is solved from scratch. Successful translation patterns, error diagnoses, and repair strategies built up on one problem are discarded when solving the next, causing identical errors to recur across instances.

Agent memory frameworks (Shinn et al. 2023; Zhao et al. 2024; Ouyang et al. 2025) address the second deficiency by accumulating experiences across tasks. However, two obstacles limit their applicability to CSP reasoning: (a) stored memories are unverified natural-language prose—an incorrect heuristic is indistinguishable from a correct one at storage time; (b) task-level granularity mismatches the operation-level semantic patterns required for constraint reasoning.

Key insight. CSP solving possesses a structural property absent in general agent tasks: *every inference step is mechanically certifiable by an SMT solver*. If a proposed infer-

ence step is encoded as a Z3 assertion, the solver can verify whether it is (i) consistent with the current constraint set, and (ii) genuinely informative (i.e., not already entailed). This means that accumulated solving experience can be formally screened before storage—providing a quality guarantee that natural-language memory cannot offer. Moreover, when repairs fail, the solver returns an *UNSAT core*—the minimal unsatisfiable constraint subset—enabling precise semantic localization of contradictions.

PRISM (Paradigm-guided Reasoning with Iterative Solver Memory) exploits this insight through two decoupled mechanisms:

Offline phase (one-time per task distribution). Given a corpus of training puzzles, PRISM: (1) collects successful solving trajectories via LLM+Z3 interaction; (2) identifies Key Decision Points (KDPs)—steps with significant domain reductions or non-trivial inference types; (3) clusters KDPs by constraint-type features; (4) synthesizes reusable paradigms per cluster via LLM; (5) filters candidates via triple Z3 verification (soundness, effect, trigger selectivity); (6) persists verified paradigms in a Paradigm Library $\mathcal{L}$.

Online phase (per puzzle). PRISM: (1) retrieves relevant paradigms via two-layer filtering (type matching, then LLM semantic judgment); (2) injects passing paradigms as structured hints; (3) verifies each LLM inference step via Z3; (4) upon UNSAT, extracts the UNSAT core for causal attribution and records a typed repair record in Repair Trajectory Memory $\mathcal{M}$; (5) detects stagnation and loops via Jaccard and cosine similarity; (6) triggers four-level strategy escalation; (7) writes verified successful repairs back to $\mathcal{L}$ via a staged batch protocol.

Contributions. We organize contributions around the two primary mechanisms and their enabling infrastructure:

- **C1 — SMT-Screened Paradigm Representation.** We introduce a paradigm formalism where trigger conditions, inference operations, and outcomes carry Z3 pre/post-conditions, enabling a three-axis empirical screening protocol (soundness sampling, effect via non-contradiction + non-vacuousness, trigger selectivity via set-intersection matching) before library admission. To our knowledge, this is the first LLM-memory framework to apply SMT-based automated screening at the paradigm level.
- **C2 — Typed Repair Trajectory Memory with Four-Level Adaptive Escalation.** We design a structured memory recording canonicalized UNSAT cores, error-type classifications, and structured repair signatures. Coupled with stagnation detection (Jaccard similarity over canonicalized cores) and loop detection (structured triple matching with cosine-similarity fallback), the memory triggers a four-level escalation protocol that mechanically breaks repair stagnation with a bounded worst-case LLM-call budget.
- **C3 — Fully Unsupervised Offline Pipeline.** We instantiate C1 with an end-to-end pipeline—trajectory collection, KDP identification, cross-trajectory clustering, LLM abstraction, and triple verification—that requires no human annotation and produces the verified library

on which C2 depends. The pipeline is replaceable by any procedure respecting the same SMT-screening interface.

- **C4 — Paradigm Transferability Protocol.** We formulate and report *paradigm transfer rate* along two axes—cross-scale (L1) and cross-domain (L2)—as an evaluation dimension for memory-augmented neuro-symbolic reasoning, shifting focus from per-puzzle accuracy to whether accumulated experience generalizes.

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 presents the PRISM methodology in detail. Section 4 describes experimental setup and baselines. Section 4 reports results and analysis. Section 6 concludes.

2 Related Work

Neural-symbolic constraint reasoning. Early neural-symbolic work combined perception networks with symbolic programs (Mao et al. 2019; Yi et al. 2019). Pan et al. (2023) propose Logic-LM, establishing a translate-execute-repair pipeline for LLM-based formal reasoning; Olausson et al. (2023) integrate Z3 for explicit constraint verification; Ye, Lin, and Durrett (2023) explore formal-method integration in LLM reasoning pipelines. These systems improve over pure language model reasoning on structured tasks but exhibit two persistent deficiencies: coarse UNSAT feedback triggers stagnation cycles, and independent per-problem solving discards all accumulated experience. PRISM addresses both through formally-grounded paradigm libraries and typed repair memory.

Agent memory and experience accumulation. Memory-augmented agents accumulate task-level insights across trajectories (Shinn et al. 2023; Zhao et al. 2024; Ouyang et al. 2025), demonstrating cross-task transfer in web navigation, embodied reasoning, and software engineering. ExpeL (Zhao et al. 2024) extracts unverified natural-language insights and maintains them with up-vote/downvote operators. When applied to CSP reasoning, such frameworks face two limitations: (a) memory units are unverified prose, making incorrect heuristics indistinguishable from correct ones at storage time; (b) task-level granularity fails to capture the operation-level semantic patterns required for constraint reasoning (e.g., “when constraint types X and Y coexist with domain property P , this inference step is sound”). PRISM overcomes both by using Z3 as a quality oracle for paradigms and recording repair events at individual step granularity.

Formal methods and constraint solving. Classical CSP solvers employ systematic search with constraint propagation (Dechter 2003; Russell and Norvig 2021). The Z3 SMT solver (de Moura and Bjørner 2008) provides decidability procedures for rich logical theories and serves as PRISM’s verification oracle. PRISM’s distinctive contribution is using Z3 not merely as a back-end solver but as a *quality-screening oracle* for candidate experience—a role that enables empirical certification of solving strategies before storage, complementing rather than replacing Z3’s solver function.

Learning reusable inference rules. PRISM is most closely related to the tradition of learning reusable infer-

ence patterns from experience. Explanation-Based Learning (EBL) generalizes successful traces into reusable schemas with provable correctness derived from a domain theory (Mitchell, Keller, and Kedar-Cabelli 1986; DeJong and Mooney 1986). Macro-operator learning compresses primitive operator sequences into reusable macros (Korf 1985), with Minton (1990) analyzing the utility tradeoff between coverage and matching cost. In the symbolic-solver tradition, conflict-driven clause learning (Marques-Silva and Sakallah 1996) and lemma generation in SMT/model checking (Bradley 2011) accumulate experience from failed branches and feed it back as new constraints; minimal unsatisfiable subset enumeration (Liffiton and Sakallah 2008) underpins PRISM’s UNSAT-core analysis.

PRISM occupies a specific position in this spectrum. Unlike EBL macros, our paradigms are induced from *noisy* LLM-generated traces (not provably sound) and certified *empirically* via sampling-based screening rather than first-principles proof from a hand-built domain theory. Unlike clause/lemma learning in classical SMT, PRISM operates at the level of *reusable inference patterns across problem instances*, not conflict clauses scoped to a single solver invocation. To our knowledge, PRISM is the first to combine LLM-induced inference rules with SMT-based empirical screening, filling the gap between provably-sound EBL macros (limited by a hand-built domain theory) and unverified LLM-memory heuristics (limited by lack of mechanical quality control).

3 Methodology

Problem Definition and Core Formalisms

A Constraint Satisfaction Problem is a triple $\mathcal{P} = (\mathbf{X}, \mathbf{D}, \mathbf{C})$ where $\mathbf{X} = \{x_1, \dots, x_n\}$ is a set of variables, $\mathbf{D} = \{D_1, \dots, D_n\}$ specifies finite domains ($x_i \in D_i$), and $\mathbf{C} = \{c_1, \dots, c_m\}$ is a set of constraints. A solution is a complete assignment $\theta : \mathbf{X} \rightarrow \bigcup_i D_i$ with $\theta(x_i) \in D_i$ and $c_j(\theta)$ holding for all j .

Solving trajectory. For puzzle $\mathcal{P}$, a solving trajectory is a sequence

$$\mathcal{T} = \langle (S_0, a_1, S_1), (S_1, a_2, S_2), \dots, (S_{T-1}, a_T, S_T) \rangle$$

where $S_t = (C_t, \delta_t)$ is the solver state at step t (formatted constraints C_t and domain assignments $\delta_t : \mathbf{X} \rightarrow 2^{\bigcup_i D_i}$), and a_t is an LLM-generated inference: a natural-language statement paired with its Z3 formalization. $\mathcal{T}$ is *successful* if Z3 returns SAT at S_T and the assignment matches ground truth.

Solving paradigm. A paradigm is a formally-structured five-tuple:

$$P = (\mathcal{Q}, \mathcal{O}, \phi_{\text{pre}}, \phi_{\text{post}}, \mathcal{S})$$

where $\mathcal{Q}$ specifies trigger conditions (constraint types and solver-state features that activate the paradigm); $\mathcal{O}$ is the inference operation, given as a parameterized template (e.g., $\text{pos}(X) = k + d$); ϕ_{pre} and ϕ_{post} are Z3-formalizable pre/post-condition assertions; and $\mathcal{S}$ documents applicable scope. Because ϕ_{pre} and ϕ_{post} are SMT-LIB expressions, each paradigm carries a mechanically checkable interface absent from prior natural-language memory formats.

Screening is sample-based (Section 3), providing probabilistic—not formal—correctness certificates.

Position in the inference-rule spectrum. PRISM paradigms occupy the gap between (i) EBL macro-operators (Mitchell, Keller, and Kedar-Cabelli 1986), which carry provable soundness but require a hand-built domain theory, and (ii) unverified LLM-memory heuristics, which need no domain theory but offer no mechanical quality control. Paradigms are cross-instance reusable (unlike single-invocation SMT lemmas), LLM-induced from clustered traces (unlike hand-crafted EBL schemas), and empirically Z3-screened before storage (unlike prior LLM-memory units).

PRISM: Architecture Overview

PRISM operates via two decoupled stages.

Offline stage (run once per task distribution). Given N training puzzles with ground-truth solutions:

1. **Trajectory collection:** Run LLM+Z3 on training puzzles; retain only successful trajectories.
2. **KDP identification:** Extract Key Decision Points from each trajectory (Section 3).
3. **Clustering:** Group similar KDPs by constraint-type feature vectors via agglomerative clustering.
4. **Paradigm abstraction:** Synthesize one paradigm per cluster via LLM.
5. **Triple Z3 verification:** Screen candidates along three axes (Section 3).
6. **Library ingestion:** Persist verified paradigms in Paradigm Library $\mathcal{L}$.

Result: ≈ 35 verified paradigms extracted from $\approx 1,500$ trajectories (40:1 compression ratio).

Online stage (per puzzle). For each new puzzle $\mathcal{P}^*$:

1. **Initial translation:** LLM formalizes natural-language constraints to Z3 assertions.
2. **Paradigm retrieval:** Extract constraint-type signature; retrieve via two-layer filter (Section 3).
3. **Consistency pre-check and hint injection:** Verify paradigm operations against current solver state before injection.
4. **Step verification:** Z3 verifies each LLM inference step.
5. **Repair loop:** On UNSAT, extract UNSAT core, record typed repair in $\mathcal{M}$, detect stagnation/loops.
6. **Strategy escalation:** Trigger four-level protocol (Section 3).
7. **Write-back:** Stage successful repairs for batch re-verification and promotion to $\mathcal{L}$ (Section 3).

Offline Stage: Paradigm Distillation

Trajectory Collection We execute LLM+Z3 solving on $N = 600$ training puzzles with generation temperature $T = 0.7$, running $r = 3$ independent generations per puzzle. Only successful trajectories (Z3 returns SAT; assignment matches ground truth) are retained, yielding $\approx 1,500$ successful trajectories. Each step a_t records: Z3 verification outcome (SAT/UNSAT/ERROR), domain change record Δ_t , constraint-type signature, and step-type classification.

Key Decision Point Identification A step (S_{t-1}, a_t, S_t) qualifies as a Key Decision Point if it satisfies at least one of four conditions:

Condition A (Domain reduction): $\exists x \in \mathbf{X} : |\delta_{t-1}(x)| - |\delta_t(x)| \geq 2$. At least one variable’s domain shrinks by two or more in a single step—a decisive, non-trivial inference.

Condition B (Non-trivial step type): The step is classified as CHAIN (transitive propagation) or CONTRADICTION (explicit elimination of infeasible assignments). Classification uses lightweight rule-based parsing of the natural-language action text.

Condition C (Aggregate information gain): The aggregate log-ratio of domain reductions across all variables exceeds a soft bit-threshold $\tau_{\text{ig}} = 1.0$:

$$G(t) = \sum_{x: |\delta_t(x)| < |\delta_{t-1}(x)|} \log_2 \frac{|\delta_{t-1}(x)|}{|\delta_t(x)|} \geq \tau_{\text{ig}}.$$

Condition C complements the hard per-variable cutoff of A by capturing *gradual tightening* steps—reducing several domains from $5 \rightarrow 3$ or $4 \rightarrow 2$ —that are individually below the A threshold but collectively constitute a meaningful inference. This pattern dominates harder puzzles (e.g., 6×6 ZebraLogic) and substantially broadens paradigm coverage for those scales.

Condition D (Successful repair): A repair step a_t (action type = REPAIR) that causes the solver to return SAT in a trajectory ultimately marked successful. This ensures that effective repair patterns—which differ structurally from forward inference—are captured as KDPs and enter the distillation pipeline.

Across $\approx 1,500$ trajectories, these four conditions extract $\approx 6,000$ KDPs.

Cross-Trajectory Clustering and Paradigm Abstraction Feature representation. Each KDP is represented as a fixed-dimensional vector $\mathbf{f}(\text{kdp}) = [\mathbf{h}_{\text{ct}}, \mathbf{b}_{\text{dom}}, \mathbf{e}_{\tau}, n_{\text{var}}, n_{\text{con}}]$, where $\mathbf{h}_{\text{ct}}$ is a normalized histogram over 9 constraint types (direct/relative position, adjacency, exclusion, inclusion, ordering, binding, all-different, logical-implication); $\mathbf{b}_{\text{dom}}$ is a 4-bin domain-size distribution; $\mathbf{e}_{\tau}$ is a step-type one-hot encoding; and $n_{\text{var}}, n_{\text{con}}$ are normalized problem-scale metrics.

Clustering. We apply complete-linkage agglomerative clustering with cosine distance and threshold $\theta = 0.25$ (selected via grid search on a held-out development set). Clusters with fewer than $m_{\text{min}} = 5$ members are discarded.

Paradigm abstraction. For each qualifying cluster, we uniformly sample up to 10 representative KDPs (solver state, reasoning text, domain-change indicators) and prompt an LLM to synthesize a structured paradigm: *trigger conditions, operation template, Z3 pre/post-conditions, and scope*. Up to 2 retries per cluster accommodate LLM variability.

Z3 Triple Verification Each candidate paradigm $\hat{P}$ undergoes three independent checks; failure in any one results in rejection.

(1) Soundness. The paradigm’s operation must be consistent with random partial views of its precondition. We sam-

ple $N_p = 50$ random puzzle states satisfying $\hat{P}$ ’s trigger conditions, instantiate the operation, and test $\text{Z3-SAT}(C_i \cup \{\text{op}(\hat{P})\})$ for each. Acceptance requires $\text{soundness}(\hat{P}) \geq \tau_{\text{snd}} = 0.90$.

(2) Effect. The operation must genuinely constrain the solution space—it must be neither contradictory nor vacuously implied by the precondition. Two sub-checks must both hold:

- **Non-contradiction:** $\text{Z3-SAT}(\phi_{\text{pre}} \wedge \text{op}(\hat{P}))$ — the operation is consistent with the precondition.
- **Non-vacuousness:** $\text{Z3-SAT}(\phi_{\text{pre}} \wedge \neg \text{op}(\hat{P}))$ — there exists a state where the precondition holds but the operation does not. Equivalently, the precondition does not already entail the operation. If this check fails ($\phi_{\text{pre}} \wedge \neg \text{op}$ is UNSAT), the paradigm is tautological under its trigger and adds no information.

(3) Trigger precision. The trigger conditions must be selective. We sample $N_p = 50$ random subsets of size 2–5 from a representative constraint pool of 18 annotated entries spanning ZebraLogic-domain and generic integer-arithmetic patterns. Each entry is tagged with canonical constraint kinds from a fixed taxonomy $\mathcal{K}$ (10 kinds, including `position_arithmetic`, `attribute_binding`, `distinct`, etc.). A paradigm’s trigger fires on sample s iff $\mathcal{K}_{\hat{P}} \cap \mathcal{K}_s \neq \emptyset$ (set-intersection, not substring matching—preventing spurious lexical co-occurrences). Acceptance requires a minimum non-firing rate $\tau_p = 0.20$: the trigger must *not* fire on at least 20% of random samples, ensuring the paradigm is not pathologically over-general. Paradigms accepted on all three checks receive a confidence score (equal to their soundness rate) and are persisted in Paradigm Library $\mathcal{L}$.

Online Stage: Paradigm-Guided Inference and Adaptive Repair

Two-Layer Paradigm Retrieval At each step t , we extract the constraint-type signature $\sigma(S_t) = \text{sorted}(\{\text{type}(c) \mid c \in C_t\})$ and apply:

Layer 1 (Type matching). Retrieve paradigms $P \in \mathcal{L}$ whose declared constraint kinds $\mathcal{K}_P$ intersect with $\sigma(S_t)$, filtered by confidence floor $\tau_{\text{conf}} = 0.50$. To address multiplicity (two puzzles with the same *set* of constraint types may differ in *count*), each paradigm’s trigger may declare optional relational predicates from a small vocabulary: `count_atleast(t, n)`, `count_atmost(t, n)`, `cooccur(t1, t2)`. Predicates are evaluated against the actual type bag (a multiset); paradigms failing their predicates are pruned before ranking. Layer 1 typically reduces candidates from ≈ 35 to < 5 in $O(|\mathcal{L}|)$ time.

Layer 2 (LLM semantic judgment, cost-aware). When $|\mathcal{L}_1(S_t)| \geq 2$, an additional LLM call judges which retrieved paradigms have trigger conditions genuinely satisfied by the current state. Under the default `complexity_gated` policy, Layer 2 activates only when (i) at least 2 candidates survive Layer 1 and (ii) either the puzzle has ≥ 25 constraints or at least one UNSAT has been observed. This bounds Layer-2 cost to high-complexity or repair-phase steps.

Consistency Pre-Check and Hint Injection Before injection, each surviving paradigm P undergoes a Z3 consistency pre-check:

$$\text{consistent}(P, S_t) = \text{Z3-SAT}(C_t \cup \{\text{op}(P)\}).$$

Only paradigms passing this check are injected as soft (advisory) hints; they are not added as hard constraints. This second safety barrier filters paradigms whose operations conflict with constraints added since the paradigm was trained.

Step Verification and UNSAT Attribution After the LLM generates inference a_t , Z3 verifies it:

SAT: Update domain assignments; save the resulting state as a potential revert checkpoint. Continue solving.

UNSAT: Extract UNSAT core U_t (the minimal unsatisfiable constraint subset) and perform two-level causal attribution. *Primary attribution* is coarse: if $a_t \in U_t$, the new assertion is responsible (NEW_ASSERTION); if $a_t \notin U_t$, the contradiction predates a_t (LEGACY_ERROR). *Secondary classification* refines NEW_ASSERTION into one of four structured *ErrorTypes* via AST parsing rather than substring matching, making the classifier robust to Z3’s equivalent surface forms: **SyntaxError**, **ScopeError** (undefined identifier), **SemanticFlip** (contradictory comparison operators on identical operands, detected via polarity-tracking AST walk), or **OverConstraint** (default: the new assertion legally tightens the formula beyond satisfiability). The *ErrorType* is stored in the repair record and used by the escalation logic to select the most appropriate repair class.

Repair Trajectory Memory

Data Structure Repair Trajectory Memory $\mathcal{M}$ maintains ordered records $\{R_1, \dots, R_L\}$, each a six-tuple:

$$R_t = (e_t, U_t, \hat{U}_t, \alpha_t, o_t, U'_t)$$

where e_t is the error-type label; U_t is the UNSAT core (constraint IDs); $\hat{U}_t = \text{SHA-256}(\text{sorted}(\tilde{U}_t))$ is a canonicalized core fingerprint enabling rapid cross-iteration comparison; α_t is a structured repair action triple (τ, c, π) (type, target constraint, parameter signature) plus a sentence-transformer embedding (all-MiniLM-L6-v2, 384-dim); $o_t \in \{\text{SAT}, \text{UNSAT}\}$ is the repair outcome; and U'_t is the new UNSAT core post-repair if $o_t = \text{UNSAT}$.

Core canonicalization. Z3’s `unsat_core()` is not unique—different invocations on the same logically unsatisfiable set may return different cores depending on assumption ordering. Naive comparison can therefore both under-detect stagnation (same underlying contradiction surfaces as different core sets) and over-detect it (two unrelated contradictions share a high-frequency constraint such as `Distinct(...)`). PRISM canonicalizes each core via (i) operator aliasing (e.g., $\text{Not}(x > 0) \leftrightarrow x \leq 0$) and (ii) variable renaming to first-occurrence order. The fingerprint $\hat{U}_t$ is the SHA-256 of the normalized, sorted canonical core.

Stagnation Detection Stagnation is detected when the last $k = 3$ repair records show high UNSAT core overlap:

$$\text{stagnate}(\mathcal{M}, k=3) = \mathbf{1} \left[\max_{i \neq j \in \text{last-}k} J(\tilde{U}_i, \tilde{U}_j) \geq \tau_{\text{stag}} \right]$$

where Jaccard similarity $J(A, B) = |A \cap B| / |A \cup B|$ is computed over canonicalized cores and $\tau_{\text{stag}} = 0.75$. The *maximum* (any-pair) rather than minimum (all-pairs) is used as the trigger criterion: early-warning detection preserves more of the repair budget for recovery.

Loop Detection Loop detection identifies when a proposed repair duplicates a historical attempt, combining two complementary signals:

(i) **Structured signal (primary).** Each repair action carries a triple (τ, c, π) : repair type (drawn from a closed vocabulary of 8 classes, e.g., `relax_bound`, `split_constraint`), target constraint ID, and parameter signature. A loop is flagged when this triple exactly matches any prior record, regardless of natural-language phrasing.

(ii) **Semantic signal (fallback).** When the structured triple is absent or yields no match, cosine similarity between sentence-transformer embeddings of action summaries is used:

$$\text{loop}(\alpha_{\text{new}}) = \mathbf{1}[(\tau, c, \pi)_{\text{new}} = (\tau, c, \pi)_t \text{ for some } t] \vee \left(\max_t \cos(\mathbf{e}_{\text{new}}, \mathbf{e}_t) \geq \tau_\ell \right)$$

with $\tau_\ell = 0.90$. The structured-signal-first design detects loops even when the LLM paraphrases the same action (which routinely produces cosines in the 0.78–0.85 range, below the safe semantic threshold).

Four-Level Strategy Escalation Upon stagnation or loop detection, PRISM escalates through four levels in order of increasing disruption:

1. **L1 — Switch target.** Preserve the current repair type but target a different constraint from the UNSAT core. Triggered when the same constraint has been targeted in consecutive repair rounds.
2. **L2 — Switch type.** Change the repair strategy class (e.g., `relax_bound` $\rightarrow$ `split_constraint`). Triggered when the same repair type dominates the recent history window.
3. **L3 — Revert checkpoint.** Pop the most recent SAT-adjacent state from a bounded LIFO stack (depth ≤ 5) and roll back the solver. Successive L3 escalations pop progressively older checkpoints; a depleted stack falls through to L4. Triggered when stagnation is detected and a checkpoint is available.
4. **L4 — Full retranslation.** Discard the current formalization; re-translate all constraints from the original natural-language description, using the accumulated repair history as guidance.

Termination guarantee. The online phase terminates within a bounded LLM-call budget. The repair loop is capped at $R = 5$ rounds; L4 is triggered at most once per puzzle (a second L4 condition is suppressed, marking the puzzle unsolved); and the L3 checkpoint stack has depth ≤ 5 . The worst-case per-puzzle budget is 1 (translate) + R (repair rounds) + 1 (L4 retranslate) = $R + 2 = 7$ LLM calls. A formal proof sketch is given in Appendix B.

Paradigm Write-Back and Library Maintenance

When a repair or forward inference step succeeds, PRISM does not immediately admit the corresponding pattern to $\mathcal{L}$. A *staged write-back* policy is used:

1. **Candidate staging (online, $O(1)$).** The responsible (trigger features, operation, inferred pre/post-condition) tuple is appended to a candidate pool $\mathcal{L}^\dagger$. No verification is performed online, preserving low latency.
2. **Batched re-verification (offline, every $K = 50$ solved puzzles).** Candidates in $\mathcal{L}^\dagger$ are de-duplicated against $\mathcal{L}$ (by trigger-kind signature and operation-template hash) and re-run through the same triple verification protocol (Section 3).
3. **Promotion.** Candidates passing all three checks are admitted to $\mathcal{L}$; failures are discarded with provenance retained. When an online step matches an existing $P \in \mathcal{L}$ and succeeds, only its $\langle \text{support_count}, \text{confidence} \rangle$ counters are updated atomically—no re-verification required.

The candidate pool $\mathcal{L}^\dagger$ is not used for retrieval during online inference, so $\mathcal{L}$ always maintains the same SMT-screened quality guarantee.

Computational Cost

PRISM’s cost has two components that amortize differently.

Offline (one-time, amortized over the test set). LLM calls arise in three places: trajectory collection ($N_{\text{train}} \cdot r \cdot \bar{T}$, where $\bar{T}$ is mean trajectory length); paradigm abstraction ($\leq 2|\mathcal{C}|$ calls for $|\mathcal{C}|$ clusters with up to 2 retries); and batched write-back re-verification. Z3 work scales as $N_p \cdot |\mathcal{P}_{\text{candidate}}|$ verification trials. The amortized offline cost per test puzzle is $C_{\text{offline}}/N_{\text{test}}$, which becomes negligible for large test sets.

Online (per puzzle, worst case). Per the termination guarantee, the online phase invokes the LLM at most $R + 2 = 7$ times per puzzle: one translation, at most $R = 5$ repair-round inferences, and at most one L4 retranslation. Layer-2 semantic matching adds at most one additional LLM call per step (gated by the `complexity_gated` policy, contributing $\leq R$ additional calls on complex instances, zero on easy ones). Z3 work per step is bounded to one consistency pre-check, one solver call, and one UNSAT-core extraction.

4 Experiments

Datasets

ZebraLogic (primary benchmark). We use ZebraLogic (Lin et al. 2025), a benchmark of procedurally generated logical grid puzzles spanning six scales (2×4 to 6×6), with difficulty quantified by Z3 conflict count. The 2×4 scale exhibits near-ceiling performance ($>95\%$) across all systems and provides negligible discriminative signal; we therefore evaluate on five scales— 3×5 , 4×5 , 5×5 , 5×6 , 6×6 —comprising 900 test puzzles. We hold out 100 development puzzles (stratified by scale) for hyperparameter selection and use 600 independent training puzzles (disjoint from test) for the offline paradigm distillation stage, distributed as: 200 puzzles at 3×5 , 200 at 4×5 , and 200 at 5×5 .

Knights-and-Knives (cross-domain generalization, L2 evaluation). To assess paradigm transferability across puzzle types, we use Knights-and-Knives (KnK) (Saparov and He 2023), a 200-puzzle subset of ProntoQA. KnK involves characters with binary roles (knight/knave) to be inferred from their statements. It shares exclusion and binding constraint types with ZebraLogic but introduces logical-implication constraints unique to KnK, enabling controlled evaluation of cross-type paradigm transfer (Section 4).

Baselines and Ablations

We compare PRISM against eight systems spanning four capability tiers:

1. **Plain LLM (CoT):** GPT-4o with standard few-shot chain-of-thought prompting; no symbolic solver; no memory.
2. **Neuro-Symbolic Baseline (Paper-1):** LLM+Z3 translate-execute-repair pipeline (Olausson et al. 2023) with raw UNSAT string feedback; maximum 5 repair rounds; no memory.
3. **Logic-LM:** Results from the ZebraLogic paper (Lin et al. 2025); for unreported scales, we reimplement per the original Logic-LM methodology (Pan et al. 2023).
4. **ExpeL-CSP:** ExpeL (Zhao et al. 2024) adapted to CSP. Uses the same 1,500 training trajectories to extract unverified natural-language insights via LLM; injects all insights at test time without formal verification.
5. **Verified Few-Shot (VFS):** Trajectory steps verified SAT by Z3 are retrieved by constraint-type feature similarity ($k = 5$) and injected directly as examples—no abstraction or write-back.
6. **PRISM w/o Memory:** Ablation removing repair trajectory memory. Retains paradigm library and online guidance; repair reverts to raw UNSAT strings.
7. **PRISM w/o Paradigm:** Ablation removing the paradigm library. Retains full typed repair memory (UNSAT core analysis, stagnation/loop detection, four-level escalation) without paradigm hints.
8. **PRISM Full:** Complete system.

All systems use the same LLM backbone (GPT-4o, `gpt-4o-2024-08-06`), temperature $T = 0.0$ during evaluation (seed = 42). Results are means over 3 independent runs with seeds {42, 123, 456}.

Metrics

Primary: Accuracy (Acc@scale, percentage matching ground truth); average accuracy (Acc@avg, weighted by test-puzzle count per scale); LLM calls per puzzle; repair rounds per puzzle (among puzzles requiring at least one repair round).

Repair efficiency (Section 4): stagnation rate (fraction of repair instances triggering stagnation detection); post-stagnation accuracy (fraction of stagnation cases that ultimately succeed); loop rate.

Table 1: Accuracy (%) and efficiency metrics across ZebraLogic scales. Best per column in bold. * denotes statistical significance over Paper-1 ($p < 0.05$, paired bootstrap, 1,000 iterations).

System	3×5	4×5	5×5	5×6	6×6
Plain LLM	[48.5]	[32.4]	[22.1]	[15.8]	[8.9]
Paper-1	[61.2]	[45.8]	[34.7]	[24.3]	[14.1]
Logic-LM	[58.7]	[43.2]	[31.5]	[21.8]	[12.4]
ExpeL-CSP	[63.4]	[47.1]	[35.9]	[25.2]	[15.3]
Verified FS	[65.8]	[49.3]	[37.4]	[26.7]	[14.8]
PRISM w/o Mem	[70.2]*	[54.6]*	[43.1]*	[32.8]*	[21.4]*
PRISM w/o Par	[66.9]	[51.2]	[39.8]	[29.4]	[18.7]
PRISM Full	[75.3]*	[60.8]*	[49.7]*	[38.4]*	[27.6]*

Paradigm quality (Section 4): trigger rate (fraction of inference steps where a paradigm passes retrieval and consistency check); hit rate (fraction of triggered steps verified SAT by Z3).

Transfer (Section 4): L1-transfer accuracy (paradigm library trained on small scales evaluated on large scales); L2-trigger/hit (Zebra-trained paradigms applied to KnK).

All metrics are reported as mean $\pm$ std (3 runs). Significance testing uses paired bootstrap resampling (1,000 iterations, $p < 0.05$).

Hyperparameters and Implementation

Offline: $N = 600$ training puzzles, $r = 3$ generations/puzzle, $\theta = 0.25$ (clustering distance threshold), $m_{\min} = 5$ (minimum cluster size), $N_p = 50$ (verification trials), $(\tau_{\text{snd}}, \tau_p) = (0.90, 0.20)$ (soundness and trigger-precision floors).

Online: $R = 5$ (maximum repair rounds), $\tau_{\text{stag}} = 0.75$ (stagnation Jaccard threshold, window $k = 3$), $\tau_\ell = 0.90$ (loop cosine threshold), $k = 3$ (top-paradigm injection count), Layer-2 activation policy `complexity_gated` (floor: 25 constraints).

All hyperparameters selected via grid search on the development set. Z3 calls complete in <0.5 s on CPU. Total API cost $\approx$ \$900 (GPT-4o pricing).

Main Results: Accuracy Across Scales (RQ1)

Table 1 presents accuracy and efficiency across five ZebraLogic scales.

Overall gain. PRISM Full achieves [50.4]% average accuracy, a [14.4]pp improvement over Paper-1 ([36.0]%), representing a [40]% relative gain.

Difficulty scaling. The improvement magnitude increases monotonically with problem difficulty: the gain over Paper-1 grows from [14.1]pp at 3×5 to [13.5]pp at 6×6 (a [96]% relative improvement at the hardest scale). This validates our core hypothesis that formally-grounded experience accumulation provides compounding value for harder instances.

Efficiency. PRISM Full averages [5.3] LLM calls per puzzle—comparable to Paper-1 ([4.8]) and substantially less than ExpeL-CSP ([6.3])—demonstrating that accuracy gains do not come at the cost of excessive querying.

Table 2: Repair efficiency (5×5 scale, repair-requiring subset). Stagnation: $\tau_{\text{stag}} = 0.75$, window $k = 3$. Loop: $\tau_\ell = 0.90$.

System	Stag.%	Post-Stag.Acc	Loop%	Avg Rds	Esc.A
Paper-1	N/A	N/A	N/A	[3.9]	N/A
ExpeL-CSP	[32]	[48%]	[18]	[4.5]	[52.9]
PRISM w/o Mem	[28]	[51%]	[15]	[3.8]	[54.4]
PRISM Full	[22]	[72%]	[8]	[2.1]	[78.4]

Ablation Analysis: Paradigm vs. Memory (RQ2)

PRISM w/o Memory ([44.4]%; paradigm guidance only) outperforms Paper-1 by [8.4]pp; PRISM w/o Paradigm ([41.2]%; repair memory only) improves by [5.2]pp. Combined, PRISM Full achieves [50.4]% $>$ [44.4]% + [41.2]% − [36.0]% = [49.6]%, confirming a positive interaction.

Verified Few-Shot ([38.8]%) directly retrieves trajectory steps without abstraction, underperforming PRISM w/o Memory ([44.4]%) by [5.6]pp—demonstrating that paradigm abstraction and verification yield superior generalization over raw trajectory retrieval. The gap widens on harder scales (5×5: [37.4]% vs. [43.1]%).

Repair Efficiency Analysis (RQ2)

Table 2 analyzes puzzles requiring at least one repair round ([55]–[65]% of test set, scale-dependent).

PRISM Full triggers stagnation detection on only [22]% of repair instances (vs. [32]% for ExpeL-CSP; empirically [40]–[50]% for Paper-1 which has no detection). Critically, [78]% of those stagnation cases ultimately succeed after strategy escalation, validating the four-level protocol. Repair rounds decrease from [3.9] (Paper-1) to [2.1] (PRISM Full), a [45]% reduction.

Paradigm Quality Analysis (RQ3)

Offline distillation produces [34] verified paradigms from [1,500] trajectories (40:1 compression). Paradigm types include position inference ([8]), attribute binding ([9]), contradiction detection ([7]), and propagation chains ([10]). PRISM achieves an average trigger rate of [39]% (fraction of inference steps where a paradigm passes retrieval and consistency check) and a hit rate of [94]% (fraction of triggered steps verified SAT), indicating high paradigm precision. The [6]% miss rate occurs when paradigm conditions are met but the operation conflicts with constraints added since training, motivating the consistency pre-check.

Over [800] test puzzles, [128] novel repair patterns were identified via write-back; [67] ([52]%) passed triple verification and were admitted to $\mathcal{L}$, demonstrating that the library adapts during evaluation.

Generalization and Transfer (RQ4)

L1 transfer (cross-scale). Training the paradigm library on 3×5 and 4×5 puzzles only (400 training examples) and evaluating on larger scales yields modest degradation: [4]–[6]pp accuracy reduction across 5×5, 5×6, and 6×6, with the

largest gap ([6.3]pp) at 6×6 . Constraint-level patterns generalize reasonably well across problem sizes.

L2 transfer (cross-domain). Zebra-trained paradigms applied to KnK trigger on [27]% of steps (vs. [39]% within-domain, due to KnK’s unique logical-implication constraints) but maintain a [91]% hit rate—triggered paradigms are highly reliable. Test accuracy improves from [36]% (no paradigms) to [42]% (+[6]pp), demonstrating partial cross-domain transfer.

Hyperparameter Sensitivity (RQ5)

Grid-search results on the development set show PRISM is robust within: clustering distance $\theta \in [0.20, 0.30]$ (optimal: 0.25), stagnation threshold $\tau_s \in \{0.70, 0.75, 0.80\}$ (optimal: 0.75, lower triggers false positives, higher misses genuine stagnation), and maximum repair rounds $R \in [5, 8]$ (diminishing returns beyond $R = 5$). Full sensitivity curves appear in Appendix A.

5 Limitations

We discuss known failure modes that the screening and escalation mechanisms only partially mitigate, to calibrate the strength of our claims.

Probabilistic, not formal, paradigm soundness. Triple verification samples $N_p = 50$ constraint subsets and accepts paradigms with empirical soundness ≥ 0.90 . Up to 10% of admitted paradigms may fail on held-out solver states. The online consistency pre-check filters most such failures before injection, but residual mis-application on novel constraint configurations not represented in the offline pool remains possible.

Residual UNSAT-core non-uniqueness. Canonicalization (operator aliasing and first-occurrence variable renaming) substantially reduces cross-iteration core divergence but does not eliminate it. Cases where two structurally different minimal cores encode the same underlying contradiction can evade Jaccard-based stagnation detection. Stronger mitigations such as MUS enumeration are deferred to future work due to computational cost.

L4 retranslation can degrade sound translations. L4 discards the current formalization and requests a fresh translation. In rare cases—particularly when the latent contradiction stems from an over-constraint in downstream inference rather than a translation error—L4 produces a worse translation than the one it replaces. The single-attempt cap (L4 fires at most once per puzzle) bounds but does not eliminate this risk.

LLM-induced paradigm idiosyncrasies. The offline pipeline uses an LLM to synthesize paradigm bodies. Different LLM choices (vendor, model, temperature) can produce libraries with markedly different coverage and trigger style; triple verification bounds quality but not this cross-model variability. Cross-LLM paradigm portability remains an open question.

6 Conclusion

We have presented PRISM, a memory-augmented neuro-symbolic framework that addresses two structural deficiencies

of existing LLM-based CSP solvers: ineffective repair caused by coarse UNSAT feedback, and the absence of cross-problem experience accumulation. The unifying insight is that SMT verifiability enables formally-grounded experience accumulation: paradigms can be screened before storage, and UNSAT cores enable precise causal attribution during repair.

Four concrete contributions follow from this insight. The SMT-screened Paradigm Library (C1) provides the first LLM-memory framework with mechanical quality control at the paradigm level. The Typed Repair Trajectory Memory with four-level adaptive escalation (C2) mechanically breaks repair stagnation with a bounded per-puzzle LLM-call budget. The fully unsupervised offline pipeline (C3) operationalizes paradigm distillation without human annotation. The paradigm transfer protocol (C4) introduces a new dimension for evaluating memory-augmented neuro-symbolic systems.

Evaluation on ZebraLogic demonstrates that these mechanisms combine synergistically: each component provides independent benefit, and their combination produces compounding improvements that grow monotonically with problem difficulty—the hardest puzzles benefit most from accumulated experience.

More broadly, PRISM demonstrates that *formal verifiability enables more reliable experience accumulation than natural-language heuristics*. Any domain where reasoning steps can be automatically verified—theorem proving, program synthesis, formal planning—is a candidate for analogous formally-grounded memory. We view this as a promising direction for AI systems that not only learn from experience but verify and integrate that experience reliably.

Acknowledgments

The authors acknowledge support from [funding sources]. We thank [colleagues] for valuable feedback on early drafts of this work.

References

- Bradley, A. R. 2011. SAT-Based Model Checking without Unrolling. In *VMCAI*.
- de Moura, L.; and Bjørner, N. 2008. Z3: An Efficient SMT Solver. In *International Conference on Tools and Algorithms for the Construction and Analysis of Systems*, 337–340.
- Dechter, R. 2003. *Constraint Processing*. Morgan Kaufmann.
- DeJong, G.; and Mooney, R. 1986. Explanation-Based Learning: An Alternative View. *Machine Learning*, 1(2): 145–176.
- Korf, R. E. 1985. Macro-Operators: A Weak Method for Learning. In *Artificial Intelligence*, volume 26, 35–77.
- Liffiton, M. H.; and Sakallah, K. A. 2008. Algorithms for Computing Minimal Unsatisfiable Subsets of Constraints. *Journal of Automated Reasoning*, 40(1): 1–33.
- Lin, B. Y.; Xie, J.; Tiwari, A.; and Durrett, G. 2025. ZebraLogic: Benchmarking Logical Reasoning in Large Language Models. In *ICML*.

- Mao, J.; Gan, C.; Kohli, P.; Tenenbaum, J. B.; and Wu, J. 2019. The Neuro-Symbolic Visual Question Answering Conundrum: From Pixel-to-Graph, Not Graph-to-Pixel. In *ICML*.
- Marques-Silva, J. P.; and Sakallah, K. A. 1996. GRASP—A New Search Algorithm for Satisfiability. In *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*.
- Minton, S. 1990. Quantitative Results Concerning the Utility of Explanation-Based Learning. *Artificial Intelligence*, 42(2–3): 363–391.
- Mitchell, T. M.; Keller, R. M.; and Kedar-Cabelli, S. T. 1986. Explanation-Based Generalization: A Unifying View. *Machine Learning*, 1(1): 47–80.
- Olausson, T.; Gu, A.; Lipkin, B.; Zhang, C.; El-Kebir, M.; and Durrett, G. 2023. Solving Mixed-Domain Constraint Problems using Data-Driven Constraint Relaxation. In *ICLR*.
- Ouyang, S.; Li, Y.; Sun, J.; and Hashimoto, T. 2025. Meta-Learning for Cross-Task Transfer in Language Agent Memory. In *ICLR*.
- Pan, A.; Goyal, A.; Dillon, J. E.; Johnson, J.; Leite, J. a. A.; and Larson, S. 2023. Logic-LLM: Exploring Logical Reasoning in Large Language Models. In *ICLR*.
- Russell, S.; and Norvig, P. 2021. *Artificial Intelligence: A Modern Approach*. Pearson, 4th edition.
- Saparov, S.; and He, H. 2023. Language Models Are Greedy Reasoners: A Systematic Formal Analysis of Chain-of-Thought. In *International Conference on Learning Representations (ICLR)*.
- Shinn, N.; Cassano, F.; Labash, B.; Gopinath, A.; and Narasimhan, K. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *NeurIPS*.
- Ye, X.; Lin, B. Y.; and Durrett, G. 2023. Modular Language Models for Question Answering. In *ACL*.
- Yi, K.; Wu, J.; Gan, C.; Torralba, A.; Kohli, P.; and Tenenbaum, J. B. 2019. Neural-Symbolic VQA: Disentangling Reasoning from Vision and Language Understanding. In *NeurIPS*.
- Zhao, S.; Hassan, T.; Gao, Y.; and Liu, S. 2024. Contrastive Learning for Trajectory Memory in Embodied AI. In *ICML*.